

แบบฝึกหัดที่ 74

1. โปรแกรมข้างล่างนี้มีที่ผิดทั้งหมด 10 คำสั่ง จงหาว่ามีคำสั่งใดบ้างที่ผิด
ห้ามตอบเกิน 10 คำสั่ง

```
#include<iostream>
using namespace std;
class C;
class A {
    int a;
protected:
    int f(A x) { a+=x.a; }
public:
    A() { a=0; }
    A(int x) { a=x; }
    A f(int x) { a=x; return *this; }
    int f() { return a; }
    A f(A x, int y) { return f(x)*y; }
    bool operator==(A &x) { return a==x.a; }
    int operator/(A x) { f(x, 1); return *this; }
    operator int() { return a; }
};
class B: public A {
    int b;
protected:
    int g(int x) { b+=x; return b; }
public:
    B() { b=2; }
    B(int b):A(b) { this->b=b; }
    B g(B y) { b=y.a+1; return *this;}
    B g(A x) { b+=f(x); return *this; }
    B g(int x, int y) {
        b=g(f(x))+y; return *this; }
    A g1() { return f(*this); }
    int g1(A &x) { return x.f(*this); }
    bool operator==(A &x) { return x==b; }
    int g2(B y) { return g(*this/y); }
    A g3(B &y) { return y.g(*this+f()); }
    A g4(B &y) { return B(y+1); }
    friend A hello(B&,C&);
};
class C: public B {
    int c;
public:
```

```
    C() { c=0; }
    C& h(B b) { c=!b; return *this; }
    C h(C c) { this->c=c; return *this; }
    B h(A *a) { return B(a->f(c).f()+1); }
    int h2(B b) { return g4(b).g1().f(); }
    A h2(C *c) { return this->c+*c; }
    int h2(A &x) { return c=g(x).g1()/(*this); }
    B h3(B y) { return g2(y).g(c, y.f()); }
    friend A hello(B&,C&);
};
A hello(B &b, C &c) {
    A a(b.g(c));
    c.c=c.b+b.b;
    cout<<c.g(a.f())+b.a<<endl;
    cout<<(a==b?b:c)<<endl;
    return a;
}
int main() {
    A a(3);
    B b(4);
    C c;
    cout<<b.g(a).hello(b,c).f()<<endl;
    cout<<c.h(&a).g(a, b).f()<<endl;
    int k;
    k=b.g1()/a;
    a=(A)c.h2(a)+3;
    a=c.h(b).g2(b)+c;
    c=c.h(b).g2(b)+c;
    C c2(c);
    k=(c2.h(b).g3(b))>10?1:2;
    k=c2.g(c, b.g2(8)).f();
    cout<<b.g4(c).f(a)<<endl;
    k=c.h2(&c2).f();
    k=c.h2(c2).f();
    B *b2 = new C;
    k=b2->g3(b).f(1);
    k=1+b2->h(c).f();
    delete b2;
}
```

2. โปรแกรมข้างล่างนี้มีคำตอบทั้งหมด 85 บรรทัด จงแสดงผลลัพธ์ทั้ง 85 บรรทัด

```
#include<iostream>
using namespace std;
class E {
    int e;
    public:
    E() { e=1; cout<<"E "<<e<<endl; }
    E(int x) {
        e=x; cout<<"EE "<<e<<endl;
    }
    virtual ~E() {
        cout<<"~E "<<e<<endl;
    }
    virtual E* plus() { e++; return this; }
    E* superPlus() {
        return plus()->plus();
    }
    void set(int i) { e=i; }
    virtual int get() { return e; }
    operator int() { return get(); }
};
```

```
class A: public E {
    int a;
    public:
    A() { a=2; cout<<"A "<<a<<endl; }
    A(int i) {
        a=i; cout<<"AA "<<a<<endl;
    }
    ~A() { cout<<"~A "<<a<<endl; }
    virtual void set(int i) { a=i; }
    E* plus() { a+=2; return this; }
    int get() { return a; }
};
```

```
class B: virtual public A {
    int b;
    public:
    B() {
        b=3; cout<<"B "<<b<<endl;
    }
    B(int i): A(6) {
        b=i; cout<<"BB "<<b<<endl;
    }
};
```

```
~B() { cout<<"~B "<<b<<endl; }
void set(int i) { b=i; }
int get() { return b; }
E* plus() { b+=3; return this; }
};
```

```
class C: virtual public A {
    int c;
    public:
    C() {
        c=4; cout<<"C "<<c<<endl;
    }
    ~C() { cout<<"~C "<<c<<endl; }
    void set(int i) { c=i; }
    int get() { return c; }
    E* plus() { c++; return this; }
    virtual void hi() {
        cout<<"hi hi"<<endl;
    }
};
```

```
class D: public C, public B {
    int d;
    public:
    D():B(7),A(6) {
        d=5; cout<<"D "<<d<<endl;
    }
    D(int i) {
        d=i; cout<<"DD "<<d<<endl;
    }
    ~D() { cout<<"~D "<<d<<endl; }
    void set(int i) { d=i; }
    int get() { return d; }
    E* plus() { d+=4; return this; }
    void hi(string s) {
        cout<<"hello "<<s<<endl;
    }
};
```

```

int main() {
    B b;
    C *c;
    A *a[3];
    D *d = new D[2];
    E **e = new E*[5];
    c = new D;
    cout<<"b.plus "<<b.plus()->get()<<endl;
    c->hi();
    delete c;
    cout<<"d[0] " <<d[0]<<endl;
    a[0] = new C;
    a[1] = &b;
    a[2] = &d[1];
    e[0] = new E(3);
    for (int i=0; i<3; i++) {
        cout<<a[i]->plus()->get()<<endl;
        a[i]->set(i);
        cout<<"*a " <<*a[i]<<endl;
        e[i+1]=a[i];
    }
    e[4] = new A(8);
    for (int i=0; i<5; i++) {
        cout<<"round " <<i<<endl;
        cout<<e[i]->E::get()<<endl;
        e[i]->set(i);
        cout<<e[i]->E::get()<<endl;
        cout<<e[i]->get()<<endl;
        cout<<*e[i]->superPlus()<<endl;
    }
    d[1].hi("OOP");
    cout<<"... Delete ..."<<endl;
    delete e[4];
    delete e[0];
    delete[] d;
    delete a[0];
    delete[] e;
    cout<<"... End ..."<<endl;
}

```

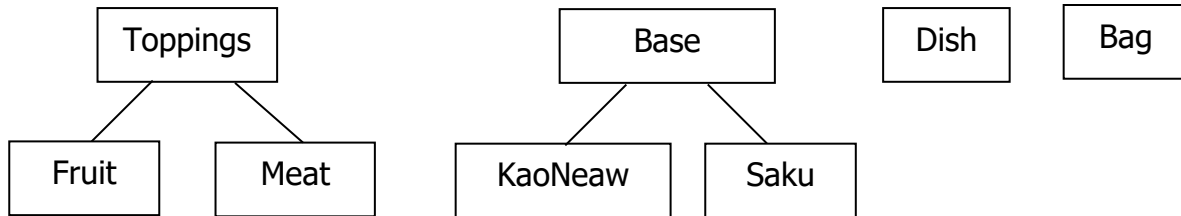
3. โปรแกรมข้างล่างนี้มีคำตอบทั้งหมด 30 บรรทัด จงแสดงผลลัพธ์ทั้ง 30 บรรทัด

```
#include<iostream>
using namespace std;
class A {
    int a;
    public:
    A() { a=5; }
    A(A &x) {
        cout<<"add one"<<endl;
        a=x.a+1;
    }
    ~A() {
        cout<<"~A "<<*this<<endl;
    }
    void set(int i) { a=i; }
    int getA() { return a; }
    A operator()(A &x) {
        cout<<"operator() "<<x<<endl;
        a += x[a];
        return *this;
    }
    int operator[](int i) { return a+i; }
    A& operator=(A &x) {
        if (this==&x) return *this;
        a=x.a+2; cout<<"hello"<<endl;
    }
    A operator++() {
        cout<<"A plus"<<endl;
        a++;
        return *this;
    }
    friend ostream& operator<<
        (ostream& os, A &a) {
            os<<"a="<<a.a;
            return os;
        }
};

class B {
    A *s;
    public:
    B() { s = new A; }
    B(B &y) {
        s = new A;
        *s=*y.s;
        cout<<"hey B"<<endl;
    }
    ~B() { delete s; cout<<"~B "<<endl; }
    A* operator[](int i) {
        s->set((*s)[i]);
        return s;
    }
    friend ostream& operator<<
        (ostream& os, B &b) {
            os<<"In B "<<*b.s;
            return os;
        }
    A getS() { return *s; }
    void setS(A x) { *s=x; }
    B operator++() {
        cout<<"B plus"<<endl;
        ++*s;
        return *this;
    }
};

int main() {
    A a1;
    A a2(a1);
    cout<<a1(a2)[6]<<endl;
    a1.set(4);
    B b1;
    B b2(b1);
    b1.setS(a1);
    cout<<b1<<endl;
    cout<<(++b1).getS().getA()<<endl;
    cout<<"b2 "<<b2[2]->getA()<<endl;
    cout<<"Good bye OOP"<<endl;
}
```

จงสร้างคลาสตามรูปด้านล่าง โดยมีการรับทอดตามรูปและไม่เกิดความกำกวม โดยกำหนดให้การรับทอดทั้งหมดเป็นการรับทอดแบบ public ทุกคลาสให้เก็บแอททริบิวต์ทั้งหมดไว้ในส่วนของ private และกำหนดให้ Toppings และ Base เป็น Abstract class โดยให้สร้างฟังก์ชัน string getType() และ void setType(int) เป็น pure virtual function



แต่ละคลาสประกอบด้วยแอททริบิวต์และความหมาย ดังนี้ (**ห้าม**เปลี่ยนชื่อแอททริบิวต์)

	แอททริบิวต์	ความหมาย
Toppings	double price;	ราคา
Fruit	int fruitType;	1.ข้าวโพด 2.ลำไย 3.เฟือง 4.มะพร้าวอ่อน 5.มะพร้าวขูด
Meat	int meatType;	1.ปลาแห้ง 2.หมูฝอย 3.หมูชิ้น 4.เนื้อฝอย
Base	double price;	ราคา
KaoNeaw	int kaoNeawType; int kaoNeawColor;	1.ข้าวเหนียวหนึ่ง 2.ข้าวเหนียวมน 3.ข้าวเหนียวเปียก 1.ขาว 2.ดำ
Saku	int sakuType; int sakuColor;	1.สาคุเปียก 2.สาคุบวช 3.สาคุนึ่ง 1.ขาว 2.เขียว 3.น้ำตาล 4.ม่วง 5.ฟ้า 6.ชมพู
Dish	string name; int dishType; Base *base; Toppings **toppings; int num;	ชื่ออาหาร 1.กล่องพลาสติก 2.ถุงพลาสติก 3.ถ้วย 4.ใบตอง พอยเตอร์ที่ชี้ไปยัง Base อาร์เรย์ขนาด num เก็บพอยเตอร์ที่ชี้ไปยัง Toppings ขนาดของอาร์เรย์ toppings
Bag	Dish **dish; int num; int bagType;	อาร์เรย์ขนาด num เก็บพอยเตอร์ที่ชี้ไปยัง Dish ขนาดของอาร์เรย์ dish 1.ถุงกระดาษ 2.ถุงพลาสติก

จงสร้างฟังก์ชันต่างๆ ในส่วนของ public ดังนี้ (**ห้าม**สร้างฟังก์ชันเพิ่มเติมถ้าโจทย์ไม่ได้สั่ง)

ทุกคลาส จงสร้าง

- default constructor ในการกำหนดค่าให้กับแอททริบิวต์ทุกตัว โดยมีการกำหนดค่าเริ่มต้นดังนี้
 - Toppings ราคา 5
 - Fruit มะพร้าวอ่อน ราคา 6
 - Meat หมูฝอย ราคา 25
 - Base ราคา 5
 - KaoNeaw ข้าวเหนียวขาวหนึ่ง ราคา 5
 - Saku สาคุเปียกสีขาว ราคา 5
 - Dish ชื่อ"no name" ใส่ใบตอง base=0 toppings=0 num=0
 - Bag ให้ dish=0 num=0 ถุงพลาสติก

- สร้างฟังก์ชัน set ในการกำหนดค่าให้กับแอทริบิวต์แต่ละตัว
 - o กรณี price ราคาต้องมากกว่าหรือเท่ากับศูนย์
 - o กรณี toppings กับ dish ให้สร้างฟังก์ชัน setToppings(int, Toppings*) setDish(int, Dish*) ในการกำหนดค่าให้กับ toppings และ dish โดยให้กำหนดค่าในอาร์เรย์ที่ละ 1 ค่า โดยระบุตำแหน่งที่เก็บในอาร์เรย์
 - o กรณี num ค่าต้องมากกว่าหรือเท่ากับศูนย์ และให้คืนพื้นที่เดิมและจองพื้นที่ใหม่ให้กับ toppings และ dish ด้วย
 - o กรณี type ต่างๆ ให้สร้าง void setType(int) โดยกำหนดค่าให้อยู่ในขอบเขตที่กำหนดตามคำอธิบาย โดยในคลาส Toppings และ Base ให้ทำเป็น pure virtual function เพื่อจะได้ทำ dynamic binding ได้
- ฟังก์ชัน get ในการรีเทิร์นแอทริบิวต์แต่ละค่า เช่น getPrice()
 - o สำหรับ toppings และ dish ให้รีเทิร์นค่าสมาชิกในอาร์เรย์ที่ละ 1 ค่าโดยระบุตำแหน่งที่เก็บในอาร์เรย์
 - o กรณี type ต่างๆ ให้สร้าง string getType() โดยที่คลาส Toppings และ Base ให้สร้าง เป็น pure virtual function เพื่อจะได้ทำ dynamic binding ได้ ส่วนในคลาสอื่นๆ ให้รีเทิร์นความหมายของ type เช่น ถ้า fruitType มีค่าเท่ากับ 1 ก็ให้รีเทิร์น "ข้าวโพด" หรือ "Corn" เป็นต้น

คลาส Toppings, Fruit, Meat จงสร้าง

- constructor โดยรับข้อมูลจากพารามิเตอร์ในการกำหนดค่าให้กับแอทริบิวต์ทุกตัวรวมทั้ง base class โดยใช้วิธีการไปเรียกใช้ constructor ของ base class และให้ใช้เงื่อนไขเดียวกันกับฟังก์ชัน set
- copy constructor ในการก๊อปปี้ค่าทุกค่าอย่างถูกต้อง

คลาส Fruit จงสร้าง

- ฟังก์ชันในการนับจำนวนอ็อบเจกต์ของคลาส Fruit ว่า ณ ปัจจุบันมีจำนวนอ็อบเจกต์คงอยู่ในโปรแกรมทั้งหมดเท่าไร ถึงแม้ว่าจะไม่มีอ็อบเจกต์อยู่เลยก็ต้องสามารถบอกได้ด้วย ให้สร้างแอทริบิวต์และฟังก์ชันที่เกี่ยวข้องเพิ่มเติมได้

คลาส Dish, Bag จงสร้าง

- ฟังก์ชัน show() โดยให้แสดงข้อมูลทุกอย่างออกทางจอภาพ
- operator << ในการเรียกใช้ show()
- destructor ให้ไปเรียกฟังก์ชัน show และ คืนพื้นที่ของแอทริบิวต์ที่มีการจอง

คลาส Dish จงสร้าง

- operator!() ในการรีเทิร์นค่าผลรวมของราคา base และ toppings ทั้งหมด
- ฟังก์ชันในการกำหนดค่าให้กับแอทริบิวต์ต่างๆ อย่างถูกต้องและมีความหมาย โดยใช้หลักการ function overloading ทั้งหมด 3 ฟังก์ชัน
- void operator=(Dish&) ในการกำหนดค่าทุกค่าและจองพื้นที่อย่างถูกต้อง
- ฟังก์ชัน bool hasTopping(string) ที่รับพารามิเตอร์เป็นชื่อชนิดของ toppings เช่น "ข้าวโพด" มาตรวจสอบว่าอ็อบเจกต์ของ Dish ใส่ toppings นี้หรือไม่

คลาส Bag จงสร้าง

- ฟังก์ชัน int dishToppings(string) ที่รับพารามิเตอร์เป็นชื่อชนิดของ toppings เช่น "ข้าวโพด" มาตรวจสอบว่าถุงใบนี้มี dish ที่ใส่ toppings นี้อยู่กี่ dish
- operator!() ในการรีเทิร์นค่าผลรวมของราคาอาหารทั้งหมดในถุง

ใน main

- จงสร้าง
 - o fruit เป็นอาร์เรย์ขนาด 5 เก็บอ็อบเจกต์ของ Fruit จากนั้นให้กำหนดค่าเป็น ข้าวโพด ลำไย ฝรั่ง มะพร้าวอ่อน มะพร้าวหูด และกำหนดราคา 5 บาททุกอัน
 - o meat เป็นพอยเตอร์ที่ชี้ไปยังคลาส Meat จากนั้นให้จองพื้นที่เป็นอาร์เรย์ขนาด 4 เก็บอ็อบเจกต์ของคลาส Meat โดยกำหนดค่าตามลำดับดังนี้ ปลาแห้ง หมูฝอย หมูชิ้น เนื้อฝอย และกำหนดราคาตามลำดับดังนี้ 10, 15, 20, 25
- จงสร้าง fruit2 เป็นอาร์เรย์ขนาด 5 เก็บพอยเตอร์ที่ชี้ไปยังคลาส Toppings โดยให้แต่ละตัวจองพื้นที่โดยสร้างอ็อบเจกต์ที่มีการไปเรียกใช้ copy constructor โดยก๊อปปี้มาจาก fruit[i], i=0, 1,..., 4 เรียงลำดับตาม index มาเรื่อยๆ จากนั้นให้เปลี่ยนราคาทั้งหมดเป็น 8 บาท และแสดงค่าของแอสทริบิวต์ทุกค่าของ fruit2 ออกทางจอภาพ
- จงสร้าง kaoNeaw เป็นอาร์เรย์ขนาด 6 เก็บอ็อบเจกต์ของ KaoNeaw โดยกำหนดค่าตามลำดับดังนี้ ข้าวเหนียวขาวนึ่ง ข้าวเหนียวขาวมูน ข้าวเหนียวขาวเปียก ข้าวเหนียวดำนึ่ง ข้าวเหนียวดำมูน ข้าวเหนียวดำเปียก แต่ละอันราคา 9 บาท จากนั้นให้แสดงรายละเอียดทุกอย่างของ kaoNeaw ออกทางจอภาพ
- จงสร้าง saku เป็นอาร์เรย์ขนาด 6 เก็บอ็อบเจกต์ของ Saku โดยกำหนดค่าตามลำดับดังนี้ สาคุเปี้ยกสีขาว สาคุบวชสีเขียว สาคุนึ่งสีน้ำตาล สาคุเปี้ยกสีม่วง สาคุบวชสีฟ้า สาคุนึ่งสีชมพู แต่ละอันราคา 8 บาท จากนั้นให้แสดงรายละเอียดทุกอย่างของ saku ออกทางจอภาพ
- จงสร้าง dish1, dish2, dish3, dish4 เป็นพอยเตอร์ที่ชี้ไปยัง Dish และให้จองพื้นที่โดยเก็บข้อมูลดังนี้
 - o dish1 ให้ชื่อ "dishMeat1" เป็นข้าวเหนียว kaoNeaw[0] ใส่ meat[0] ใส่ใบตอง
 - o dish2 ให้ชื่อ "dishMeat2" เป็นข้าวเหนียว kaoNeaw[3] ใส่ meat[1], meat[2] ใส่ถุงพลาสติก
 - o dish3 ให้ชื่อ "dishFruit3" เป็นข้าวเหนียว kaoNeaw[2] ใส่ fruit[1] fruit[2] fruit[3] ใส่กล่องพลาสติก
 - o dish4 ให้ชื่อ "dishFruit4" เป็นข้าวเหนียว kaoNeaw[5] ใส่ fruit[0] ใส่กล่องพลาสติกจากนั้นให้สร้าง bag1, bag2 เป็นพอยเตอร์ที่ชี้ไปยัง Bag และให้จองพื้นที่โดยให้เป็นถุงพลาสติก โดยที่ bag1 ให้เก็บค่า dish1 และ dish2 ส่วน bag2 ให้เก็บค่า dish3 และ dish4
- จงสร้าง dishSaku เป็นอาร์เรย์ขนาด 3 เก็บพอยเตอร์ที่ชี้ไปยัง Dish และให้สมาชิกในอาร์เรย์แต่ละตัวจองพื้นที่โดยเก็บข้อมูลดังนี้
 - o DishSaku[0] ให้ชื่อ "dishSaku5" เป็นสาคุ saku[0] ใส่ fruit2[0] fruit2[1] ใส่ถ้วย
 - o DishSaku[1] ให้ชื่อ "dishSaku6" เป็นสาคุ saku[4] ใส่ fruit2[3] ใส่ถ้วย
 - o DishSaku[2] ให้ชื่อ "dishSaku7" เป็นสาคุ saku[2] ใส่ fruit2[2] fruit2[4] ใส่ถ้วยจากนั้นให้สร้าง bag3 เป็นพอยเตอร์ที่ชี้ไปยัง Bag และให้จองพื้นที่โดยให้เป็นถุงกระดาษ เก็บค่า dishSaku แต่ละตัว
- จงสร้าง bagAll เป็นอาร์เรย์ขนาด 3 เก็บพอยเตอร์ที่ชี้ไปยัง Bag และให้สมาชิกแต่ละตัวชี้ไปที่ bag1, bag2, bag3 ตามลำดับ จากนั้นให้เรียกใช้ operator<< ที่สร้างไว้สำหรับ Bag แสดงค่าทั้งหมดของ bagAll ออกทางจอภาพ และให้แสดงราคายอดรวมทั้งหมดของอาหารในถุง bagAll
- จงหาว่าใน bagAll มีอาหารถุงไหนที่ใส่ ข้าวโพด บ้าง ถ้าถุงนั้นมีอาหารที่ใส่ข้าวโพด ให้แสดงค่าราคารวมของอาหารถุงนั้นออกทางจอภาพ
- จงเปลี่ยนราคาของ fruit[2] และ fruit2[2] เป็น 11 และ 12 ตามลำดับ จากนั้นจงหาว่าใน bagAll มีอาหารที่ใส่ ฝรั่ง อยู่กี่อัน และราคารวมของฝรั่งที่ใส่ในอาหารนั้นมีราคารวมเท่าไร ให้แสดงผลทางจอภาพ

- จงสร้าง myDish เป็นอ็อบเจกต์ของ Dish จากนั้นให้เรียกใช้ operator= ในการกำหนดค่าให้เท่ากับค่าของ dishSaku[2] จากนั้นให้เปลี่ยนชื่ออาหารเป็น myDish และให้แสดงค่าของ myDish ทั้งหมดออกทางจอภาพโดยการเรียกใช้ operator<< ที่สร้างไว้สำหรับ Dish
- จงคืนพื้นที่ในลำดับที่ถูกต้อง

เฉลย

```
#include<iostream>
using namespace std;
class Toppings {
    double price;
public:
    virtual string getType() = 0;
    virtual void setType(int) = 0;
    Toppings() {
        setPrice(5);
    }
    Toppings(double p) {
        setPrice(p);
    }
    Toppings(Toppings& t){
        setPrice(t.price);
    }
    virtual ~Toppings() {}
    void setPrice(double p) {
        price = (p >= 0) ? p : 0;
    }
    double getPrice() {
        return price;
    }
};
class Fruit: public Toppings {
    int fruitType;
    static int count;
public:
    Fruit() {
        setType(4);
        setPrice(6);
        count++;
    }
    Fruit(int t, double p): Toppings(p) {
        setType(t);
        count++;
    }
}
```

```

    Fruit(Fruit& t): Toppings(t) {
        setType(t.fruitType);
        count++;
    }
    ~Fruit() {
        count--;
    }
    void setType(int t) {
        fruitType = (t>=1 && t<=5) ? t : 1;
    }
    string getType() {
        switch(fruitType) {
            case 1: return "Corn";
            case 2: return "Longan";
            case 3: return "Taro";
            case 4: return "Young coconut";
            case 5: return "coconut kood";
        }
    }
    static int getCount() {
        return count;
    }
};
int Fruit::count;
class Meat: public Toppings {
    int meatType;
public:
    Meat() {
        setType(2);
        setPrice(25);
    }
    Meat(int t, double p): Toppings(p) {
        setType(t);
    }
    Meat(Meat& t): Toppings(t) {
        setType(t.meatType);
    }
    void setType(int t) {
        meatType = (t>=1 && t<=4) ? t : 1;
    }
    string getType() {
        switch(meatType) {
            case 1: return "Pla hang";
            case 2: return "Mu foy";

```

```

        case 3: return "Mu chin";
        case 4: return "Nuea foy";
    }
}
};
class Base {
    double price;
public:
    virtual string getType() = 0;
    virtual void setType(int) = 0;
    Base() {
        setPrice(5);
    }
    virtual ~Base() {}
    void setPrice(double p) {
        price = ( p >= 0 ) ? p : 0;
    }
    double getPrice() {
        return price;
    }
};
class KaoNeaw: public Base {
    int kaoNeawType;
    int kaoNeawColor;
public:
    KaoNeaw() {
        setType(1);
        setPrice(5);
    }
    void setType(int t) {
        kaoNeawType = ( t>=1 && t<=3 ) ? t : 1;
    }
    string getType() {
        switch(kaoNeawType) {
            case 1: return "KaoNeaw Nueng";
            case 2: return "KaoNeaw Moon";
            case 3: return "KaoNeaw Peak";
        }
    }
    void setKaoNeawColor(int t) {
        kaoNeawColor = (t>=1 && t<=2) ? t : 1;
    }
}

```

```

        int getKaoNeawColor() {
            return kaoNeawColor;
        }
};
class Saku: public Base {
    int sakuType;
    int sakuColor;
public:
    Saku() {
        setType(1);
        setPrice(5);
    }
    void setType(int t) {
        sakuType = (t>=1 && t<=3) ? t : 1;
    }
    string getType() {
        switch(sakuType) {
            case 1: return "Saku Peak";
            case 2: return "Saku Buat";
            case 3: return "Saku Nueng";
        }
    }
    void setSakuColor(int t) {
        sakuColor = ( t>=1 && t<=6 ) ? t : 1;
    }
    int getSakuColor() {
        return sakuColor;
    }
};
class Dish {
    string name;
    int dishType;
    Base *base;
    Toppings **toppings;
    int num;
public:
    Dish() {
        setName("no name");
        setType(4);
        base=0;
        toppings=0;
        num=0;
    }
};

```

```

void setName(string s) {
    name=s;
}
int setType(int t) {
    dishType = ( t>=1 && t<=4 ) ? t : 1;
}
void setBase(Base *b) {
    base = b;
}
void setToppings(int i, Toppings* t) {
    toppings[i]=t;
}
void setNum(int n) {
    if (num==n) return;
    num = (n >= 0) ? n : 0;
    delete[] toppings;
    toppings = new Toppings*[num];
}
string getName() {
    return name;
}
string getType() {
    switch(dishType) {
        case 1: return "Plastic Box";
        case 2: return "Plastic Bag";
        case 3: return "Cup";
        case 4: return "banana leaf";
    }
}
Base* getBase() {
    return base;
}
Toppings* getToppings(int i) {
    return toppings[i];
}
int getNum() {
    return num;
}
void show() {
    cout<<"Name: "<<name<<endl;
    cout<<"DishType: "<<getType()<<endl;
    cout<<"Base: "<<base->getType()<<" price "<<base->getPrice()<<endl;
    if (base->getType() == "KaoNeaw Nueng" || base->getType() == "KaoNeaw
Moon" || base->getType() == "KaoNeaw Peak")

```

```

        cout<<"color kaoNeaw " <<((KaoNeaw*)(base))->getKaoNeawColor()<<endl;
    else
        cout<<"color saku " <<((Saku*)(base))->getSakuColor()<<endl;
    cout<<"num of toppings: " <<num<<endl;
    for (int i=0; i<num; i++) {
        cout<<i+1<<" "<<toppings[i]->getType()<<" price: " <<toppings[i]-
>getPrice()<<endl;
    }
}
~Dish() {
    show();
    delete[] toppings;
}
double operator!() {
    double total=0;
    total += base->getPrice();
    for (int i=0; i<num; i++) {
        total += toppings[i]->getPrice();
    }
    return total;
}
void set(string s) {
    setName(s);
}
void set(int i) {
    setType(i);
}
void set(Base *b) {
    setBase(b);
}
void operator=(Dish &d) {
    if (this==&d) return;
    setName(d.name);
    setType(d.dishType);
    setBase(d.base);
    setNum(d.num);
    for (int i=0; i<num; i++)
        toppings[i] = d.getToppings(i);
}
bool hasTopping(string s) {
    for (int i=0; i<num; i++)
        if (s == toppings[i]->getType())
            return true;
    return false;
}

```

```

    }
};
ostream& operator<<(ostream &os, Dish &d) {
    d.show();
    return os;
}
class Bag {
    Dish **dish;
    int num;
    int bagType;
public:
    Bag() {
        setType(2);
        dish=0;
        num=0;
    }
    int setType(int t) {
        bagType = ( t>=1 && t<=2 ) ? t : 1;
    }
    void setDish(int i, Dish* t) {
        dish[i]=t;
    }
    void setNum(int n) {
        if (num==n) return;
        num = ( n>=0 ) ? n : 0;
        delete[] dish;
        dish = new Dish*[num];
    }
    string getType() {
        switch(bagType) {
            case 1: return "Paper Bag";
            case 2: return "Plastic Bag";
        }
    }
    Dish* getDish(int i) {
        return dish[i];
    }
    int getNum() {
        return num;
    }
    void show() {
        cout<<"BAG"<<endl;
        cout<<"BagType: "<<getType()<<endl;
        cout<<"Num of dish: "<<num<<endl;
    }
}

```

```

        for (int i=0; i<num; i++) {
            cout<<"--"<<i+1<<"--"<<endl;
            dish[i]->show();
        }
    }
    ~Bag() {
        show();
        delete[] dish;
    }
    int dishToppings(string s) {
        int count=0;
        for (int i=0; i<num; i++) {
            if (dish[i]->hasTopping(s))
                count++;
        }
        return count;
    }
    double operator!() {
        double total=0;
        for (int i=0; i<num; i++)
            total += !*dish[i];
        return total;
    }
};

ostream& operator<<(ostream &os, Bag &d) {
    d.show();
    return os;
}

int main() {
    cout<<"\n....1....."<<endl;
    Fruit fruit[5];
    for (int i=0; i<5; i++) {
        fruit[i].setType(i+1);
        fruit[i].setPrice(5);
    }
    Meat *meat;
    meat = new Meat[4];
    for (int i=0; i<4; i++)
        meat[i].setType(i+1);
    meat[0].setPrice(10);
    meat[1].setPrice(15);
    meat[2].setPrice(20);
    meat[3].setPrice(25);
}

```

```

cout<<"\n....2....."<<endl;
Toppings *fruit2[5] = { new Fruit(fruit[0]),
                        new Fruit(fruit[1]),
                        new Fruit(fruit[2]),
                        new Fruit(fruit[3]),
                        new Fruit(fruit[4]) };
for (int i=0; i<5; i++) {
    fruit2[i]->setPrice(8);
    cout<<fruit2[i]->getType()<<" price "<<fruit2[i]->getPrice()<<endl;
}

cout<<"\n....3....."<<endl;
KaoNeaw kaoNeaw[6];
for (int i=0; i<3; i++) {
    kaoNeaw[i].setType(i+1);
    kaoNeaw[i].setKaoNeawColor(1);
    kaoNeaw[i].setPrice(9);
    kaoNeaw[i+3].setType(i+1);
    kaoNeaw[i+3].setKaoNeawColor(2);
    kaoNeaw[i+3].setPrice(9);
}
for (int i=0; i<6; i++) {
    cout<<kaoNeaw[i].getType()<<" "<<kaoNeaw[i].getKaoNeawColor()<<"
"<<kaoNeaw[i].getPrice()<<endl;
}

cout<<"\n....4....."<<endl;
Saku saku[6];
for (int i=0; i<3; i++) {
    saku[i].setType(i+1);
    saku[i].setSakuColor(i+1);
    saku[i].setPrice(8);
    saku[i+3].setType(i+1);
    saku[i+3].setSakuColor(i+4);
    saku[i+3].setPrice(8);
}
for (int i=0; i<6; i++) {
    cout<<saku[i].getType()<<" "<<saku[i].getSakuColor()<<"
"<<saku[i].getPrice()<<endl;
}

```

```
cout<<"\n....5....."<<endl;
Dish *dish1 = new Dish;
dish1->setName("dishMeat1");
dish1->setType(4);
dish1->setBase(&kaoNeaw[0]);
dish1->setNum(1);
dish1->setToppings(0, &meat[0]);
```

```
Dish *dish2 = new Dish;
dish2->setName("dishMeat2");
dish2->setType(2);
dish2->setBase(&kaoNeaw[3]);
dish2->setNum(2);
dish2->setToppings(0, &meat[1]);
dish2->setToppings(1, &meat[2]);
```

```
Dish *dish3 = new Dish;
dish3->setName("dishFruit3");
dish3->setType(1);
dish3->setBase(&kaoNeaw[2]);
dish3->setNum(2);
dish3->setToppings(0, &fruit[1]);
dish3->setToppings(1, &fruit[2]);
dish3->setToppings(2, &fruit[3]);
```

```
Dish *dish4 = new Dish;
dish4->setName("dishFruit4");
dish4->setType(1);
dish4->setBase(&kaoNeaw[5]);
dish4->setNum(1);
dish4->setToppings(0, &fruit[0]);
```

```
Bag *bag1 = new Bag;
bag1->setType(2);
bag1->setNum(2);
bag1->setDish(0, dish1);
bag1->setDish(1, dish2);
```

```
Bag *bag2 = new Bag;
bag2->setType(2);
bag2->setNum(2);
bag2->setDish(0, dish3);
bag2->setDish(1, dish4);
```

```

cout<<"\n....6....."<<endl;
Dish *dishSaku[3];
dishSaku[0] = new Dish;
dishSaku[0]->setName("dishSaku5");
dishSaku[0]->setType(3);
dishSaku[0]->setBase(&saku[0]);
dishSaku[0]->setNum(2);
dishSaku[0]->setToppings(0, fruit2[0]);
dishSaku[0]->setToppings(1, fruit2[1]);

dishSaku[1] = new Dish;
dishSaku[1]->setName("dishSaku6");
dishSaku[1]->setType(3);
dishSaku[1]->setBase(&saku[4]);
dishSaku[1]->setNum(1);
dishSaku[1]->setToppings(0, fruit2[3]);

dishSaku[2] = new Dish;
dishSaku[2]->setName("dishSaku7");
dishSaku[2]->setType(3);
dishSaku[2]->setBase(&saku[2]);
dishSaku[2]->setNum(2);
dishSaku[2]->setToppings(0, fruit2[2]);
dishSaku[2]->setToppings(1, fruit2[4]);

Bag *bag3 = new Bag;
bag3->setType(1);
bag3->setNum(3);
bag3->setDish(0, dishSaku[0]);
bag3->setDish(1, dishSaku[1]);
bag3->setDish(2, dishSaku[2]);

cout<<"\n....7....."<<endl;

Bag *bagAll[3];
bagAll[0] = bag1;
bagAll[1] = bag2;
bagAll[2] = bag3;

double total=0;
for (int i=0; i<3; i++) {
    cout<<*bagAll[i]<<endl;
    total += !*bagAll[i];
}

```

```

cout<<"\n Total = "<<total<<endl;

cout<<"\n....8....."<<endl;
for (int i=0; i<3; i++)
    if (bagAll[i]->dishToppings("Corn"))
        cout<<"has corn "<<!*bagAll[i]<<endl;

cout<<"\n....9....."<<endl;
fruit[2].setPrice(11);
fruit2[2]->setPrice(12);
int count=0;
total=0;
for (int i=0; i<3; i++) {
    for (int j=0; j<bagAll[i]->getNum(); j++) {
        for (int k=0; k<bagAll[i]->getDish(j)->getNum(); k++) {
            if (bagAll[i]->getDish(j)->getToppings(k)->getType() == "Taro") {
                count++;
                total += bagAll[i]->getDish(j)->getToppings(k)->getPrice();
            }
        }
    }
}
cout<<"Taro "<<count<<" price: "<<total<<endl;

cout<<"\n....10....."<<endl<<endl;
Dish myDish;
myDish=*dishSaku[2];
myDish.setName("myDish");
cout<<myDish<<endl;

cout<<"\n....11....."<<endl<<endl;
cout<<"delete bag"<<endl;
delete bag3;
delete bag2;
delete bag1;
cout<<"delete dish"<<endl;
delete dish1;
delete dish2;
delete dish3;
delete dish4;
for (int i=0; i<3; i++)
    delete dishSaku[i];
cout<<"delete meat"<<endl;

```

```
delete[] meat;  
cout<<"delete fruit2"<<endl;  
for (int i=0; i<5; i++)  
    delete fruit2[i];  
cout<<"END OOP"<<endl;  
}
```